

TALLER 5 – SEMANA 5
IMPLEMENTACIÓN ABSTRACT FACTORY - PRUEBAS

OSCAR YAIR PARDO PINEDA
JORGE ALFREDO LEAL CRUZ

PRESENTADO A:
ELIECER ONTERO OJEDA

GRUPO: E196

INGENIERIA DE SISTEMAS



INSTITUCION UNIVERSITARIA TECNOLOGICA DE SANTANDER
BUCARAMANGA

2026 – 1

PROYECTO A EJECUTARSE

PROYECTO SECTOR LOGÍSTICO/TRANSPORTE

Sistema de Gestión de Flotas

▼ Autores

Estudiante 1: Oscar Yair Pardo Pineda

Estudiante 2: Jorge Alfredo Leal Cruz

Docente: Eliecer Montero Ojeda

Unidades Tecnológicas de Santander - 2026

SpringBoot 3.x

Java 17

MySQL Database

Status En Desarrollo

Proyecto Académico

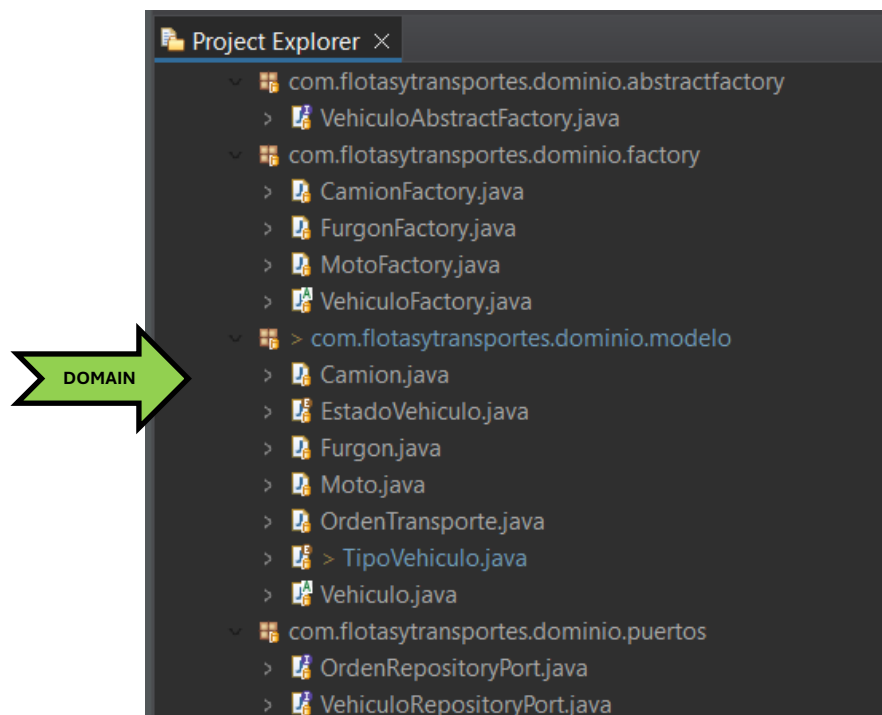
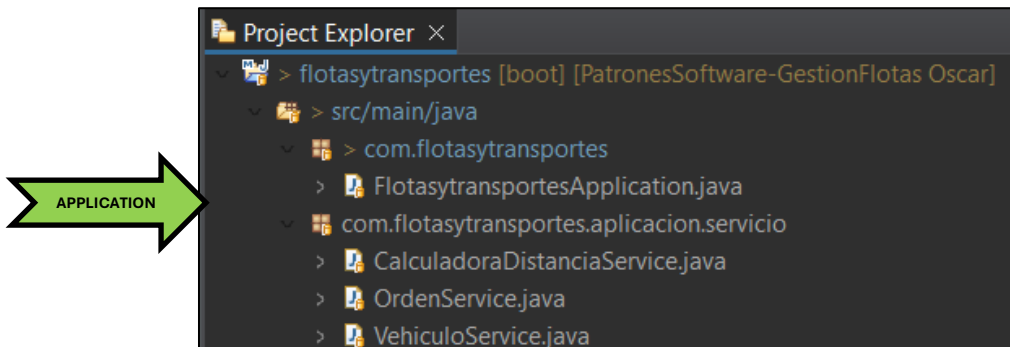
UTS Logística

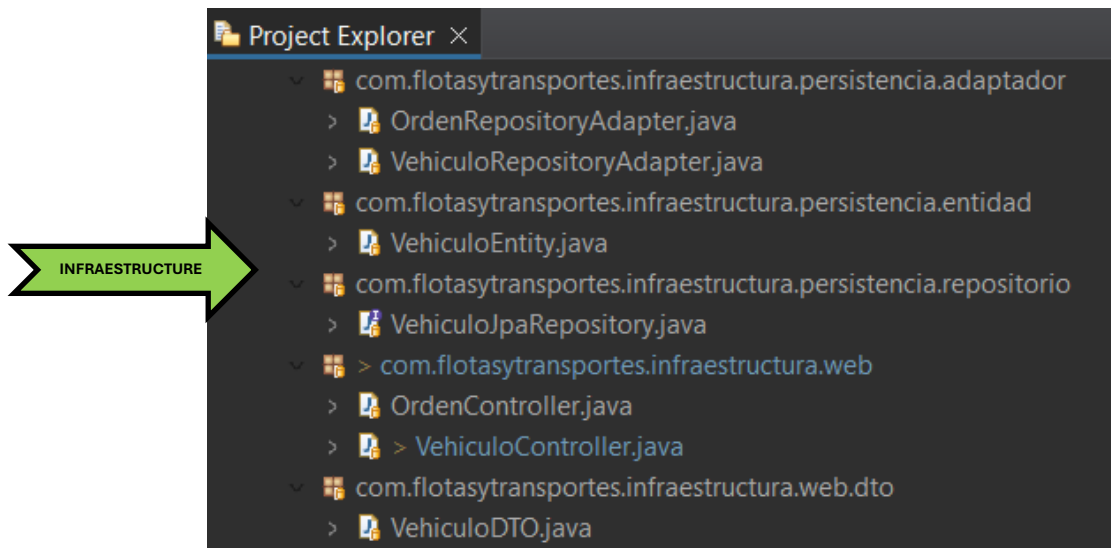
SOLID

Principios

Sistema Gestión de Flotas

ESTRUCTURA DEL PROYECTO





PATRÓN ABSTRACT FACTORY

El **ABSTRACT FACTORY** es un patrón de diseño creacional cuyo propósito es proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón permite que el código cliente utilice diferentes conjuntos de objetos compatibles entre sí sin quedar acoplado a sus implementaciones específicas.

La idea principal del patrón es separar la lógica de creación de objetos de su uso, delegando dicha responsabilidad a una fábrica abstracta que define los métodos de creación, mientras que las fábricas concretas implementan esos métodos para instanciar los productos específicos. Este patrón es especialmente útil cuando un sistema necesita trabajar con múltiples familias de objetos que deben utilizarse conjuntamente, garantizando que los objetos creados sean compatibles entre sí.

Uno de los aspectos importantes del **ABSTRACT FACTORY** es que generalmente utiliza internamente el patrón Factory Method para realizar la instanciación de los productos concretos. Mientras que el **FACTORY METHOD** se enfoca en la creación de un solo producto, el **ABSTRACT FACTORY** coordina la creación de conjuntos completos de objetos relacionados.

Entre las principales ventajas de este patrón se encuentran el desacoplamiento entre el cliente y las clases concretas, la facilidad para cambiar o ampliar familias de productos y el cumplimiento del principio Open/Closed, ya que es posible agregar nuevas familias de objetos sin modificar el código existente.

Sin embargo, también presenta algunas desventajas, como el aumento en el número de clases dentro del sistema y la dificultad para agregar nuevos tipos de productos si no fueron contemplados inicialmente en la interfaz de la fábrica. En sistemas complejos o escalables, el patrón **ABSTRACT FACTORY** resulta especialmente útil para mantener una arquitectura organizada, flexible y extensible, permitiendo que la creación de objetos se gestione de manera centralizada y coherente con el dominio del sistema.

IMPLEMENTACIÓN

En el proyecto se implementa el patrón **ABSTRACT FACTORY** para gestionar la creación de los distintos tipos de vehículos dentro de la flota.

La clase abstracta definida en el dominio es:

```
VehiculoAbstractFactory.java ×
1 package com.flotasytransportes.dominio.abstractfactory;
2
3 import com.flotasytransportes.dominio.modelo.Vehiculo;
4
5
6 public interface VehiculoAbstractFactory {
7
8     Vehiculo crearVehiculo(String placa, Double latitud, Double longitud, EstadoVehiculo estado,
9                             Double kilometrajeActual, Double limiteMantenimiento);
10
11 }
```

Esta interfaz establece el contrato que deben cumplir todas las clases encargadas de crear vehículos dentro del sistema.

A partir de esta clase abstracta se implementan las siguientes clases concretas:

- CamionFactory
- FurgonFactory
- MotoFactory

Cada una de estas clases se encarga de crear un tipo específico de vehículo.

```
VehiculoFactory.java ×
1 package com.flotasytransportes.dominio.factory;
2
3 import com.flotasytransportes.dominio.abstractfactory.VehiculoAbstractFactory;
4
5
6 public abstract class VehiculoFactory implements VehiculoAbstractFactory {
7
8     // ESTE ES EL FACTORY METHOD
9     public abstract Vehiculo crearVehiculo(
10         String placa,
11         Double latitud,
12         Double longitud,
13         EstadoVehiculo estado,
14         Double kilometrajeActual,
15         Double limiteMantenimiento
16     );
17 }
```

En esta estructura se observa la aplicación del patrón **ABSTRACT FACTORY** para organizar la creación de los distintos tipos de vehículos del sistema. La interfaz **VehiculoAbstractFactory** actúa como la clase abstracta, ya que define el método **crearVehiculo ()** que deben implementar todas las fábricas encargadas de crear objetos del dominio.

A partir de esta abstracción se implementan las clases concretas, como **CamionFactory**, **FurgonFactory** y **MotoFactory**. Cada una de estas clases se especializa en la creación de un tipo específico de vehículo. Por ejemplo, **CamionFactory** implementa el método **crearVehiculo ()** y retorna una instancia de Camión, encapsulando la lógica necesaria para construir ese objeto.

Los objetos que finalmente se crean (Camión, Furgon y Moto) corresponden a los productos concretos del patrón. Todos ellos pertenecen a la misma familia de objetos del dominio (Vehículo), pero representan diferentes tipos de vehículos dentro del sistema de gestión de flotas.

```
VehiculoService.java X
43     }
44
45     public Vehiculo crearVehiculo(TipoVehiculo tipo, VehiculoDTO dto) {
46
47         VehiculoAbstractFactory abstractfactory;
48
49         switch (tipo) {
50             case CAMION:
51                 abstractfactory = new CamionFactory();
52                 break;
53             case MOTO:
54                 abstractfactory = new MotoFactory();
55                 break;
56             case FURGON:
57                 abstractfactory = new FurgonFactory();
58                 break;
59             default:
60                 throw new IllegalArgumentException("Tipo no soportado");
61         }
62
63         Vehiculo vehiculo = abstractfactory.crearVehiculo(
64             dto.getPlaca(),
65             dto.getLatitud(),
66             dto.getLongitud(),
67             dto.getEstado(),
68             dto.getKilometrajeActual(),
69             dto.getLimiteMantenimiento()
70         );
71     }
```

Dentro de la clase **VehiculoService**, este método utiliza el patrón **ABSTRACT FACTORY** para crear diferentes tipos de vehículos de manera desacoplada.

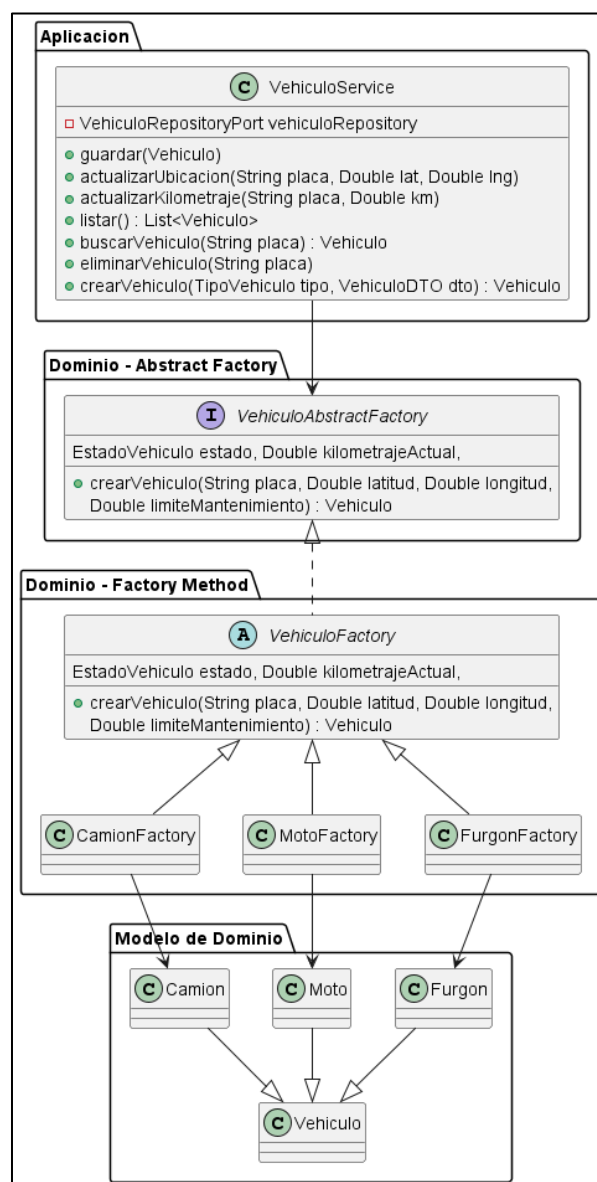
Primero, se declara una variable de tipo **VehiculoAbstractFactory**, que representa la clase abstracta encargada de definir cómo se crean los vehículos. Luego, mediante una estructura

switch, el sistema selecciona la clase concreta correspondiente según el tipo de vehículo solicitado (CamionFactory, MotoFactory o FurgonFactory).

Una vez elegida la clase, se llama al método **crearVehiculo ()** pasando los datos del **VehiculoDTO**. Cada clase concreta se encarga de crear el objeto específico (Camion, Moto o Furgon).

De esta forma, el servicio no depende directamente de las clases concretas, sino de la abstracción de la clase, lo que permite mantener el sistema desacoplado y facilita la incorporación de nuevos tipos de vehículos sin modificar la lógica principal.

UML



PRUEBAS

Esta sección presenta las pruebas integrales del módulo de vehículos, en el cual se implementa el patrón **ABSTRACT FACTORY** para la creación de distintos tipos de vehículos dentro del sistema. Las pruebas validan el correcto funcionamiento de las operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar) utilizando el endpoint /vehículos, permitiendo gestionar vehículos a través de su placa como identificador principal.

OBJETIVOS

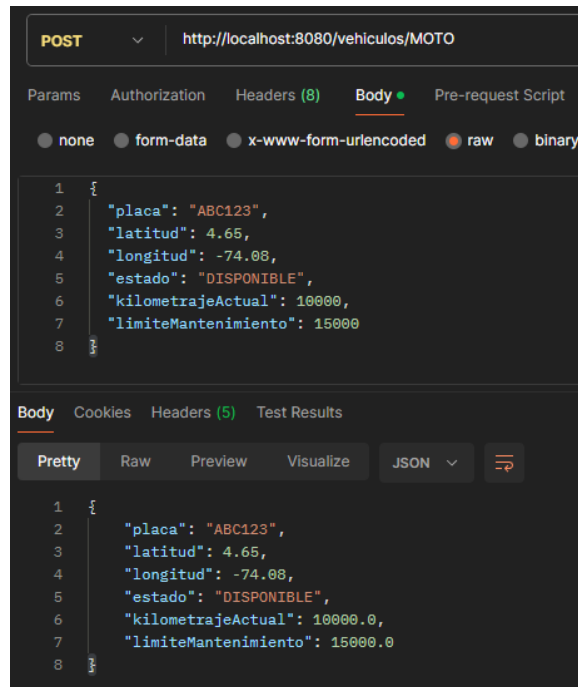
- Verificar el correcto funcionamiento de la creación y actualización de vehículos mediante las operaciones POST / PUT sobre el endpoint /vehiculos/{placa}.
- Comprobar la correcta implementación del patrón ABSTRACT FACTORY, permitiendo crear distintos tipos de vehículos a través de sus clases concretas sin acoplar el servicio a implementaciones específicas.
- Validar la consulta de información mediante GET /vehiculos para listar todos los vehículos registrados y GET /vehiculos/{placa} para la búsqueda individual.
- Confirmar la correcta eliminación de registros mediante DELETE /vehiculos/{placa}, garantizando la integridad del sistema tras la eliminación.

ALCANCE DE VALIDACIÓN DE PRUEBAS

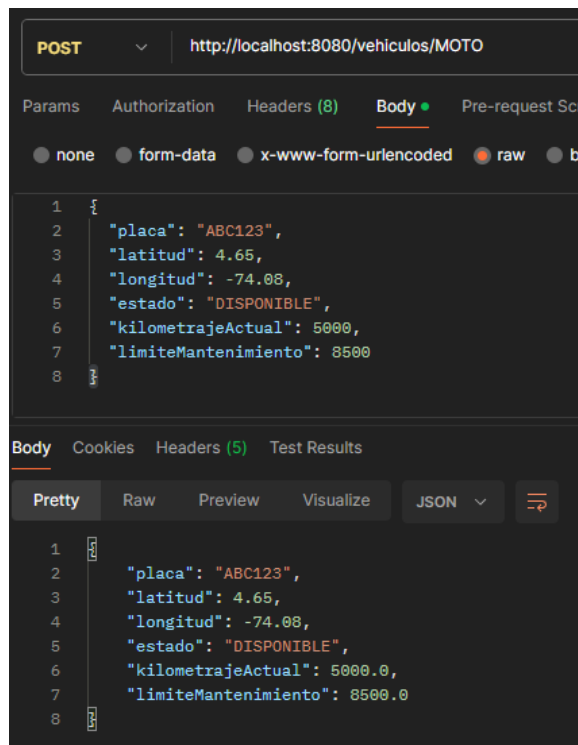
POST / PUT	vehiculos/{placa}	Creación de un vehiculo nuevo / Actualización del vehiculo
GET	vehiculos	Lista completa de vehiculos
GET	vehiculos/{placa}	Buscar un vehiculo
DELETE	vehiculos/{placa}	Eliminar un vehiculo existente

SOPORTE PRUEBAS

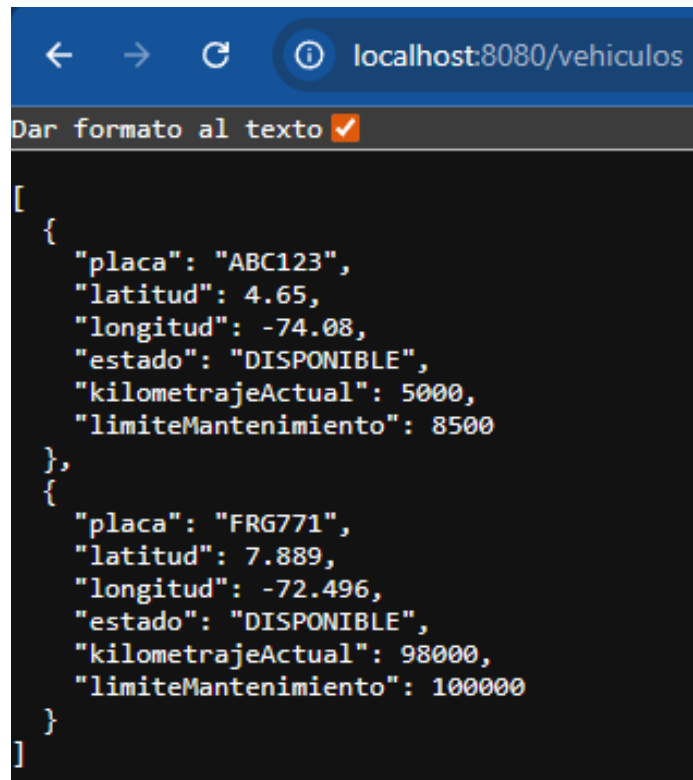
CREACIÓN DE UN VEHICULO NUEVO



ACTUALIZACION DEL VEHICULO



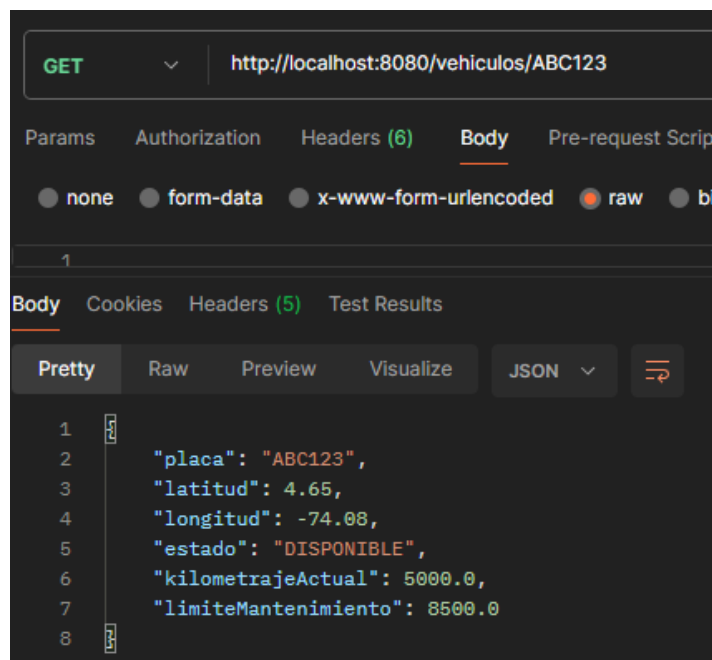
LISTA COMPLETA DE VEHICULOS



A screenshot of a web browser window. The address bar shows 'localhost:8080/vehiculos'. Below the address bar, there is a checkbox labeled 'Dar formato al texto' which is checked. The main content area displays a JSON array of two vehicle objects. The first object has a license plate 'ABC123', latitude 4.65, longitude -74.08, status 'DISPONIBLE', current mileage 5000, and maintenance limit 8500. The second object has a license plate 'FRG771', latitude 7.889, longitude -72.496, status 'DISPONIBLE', current mileage 98000, and maintenance limit 100000.

```
[
  {
    "placa": "ABC123",
    "latitud": 4.65,
    "longitud": -74.08,
    "estado": "DISPONIBLE",
    "kilometrajeActual": 5000,
    "limiteMantenimiento": 8500
  },
  {
    "placa": "FRG771",
    "latitud": 7.889,
    "longitud": -72.496,
    "estado": "DISPONIBLE",
    "kilometrajeActual": 98000,
    "limiteMantenimiento": 100000
  }
]
```

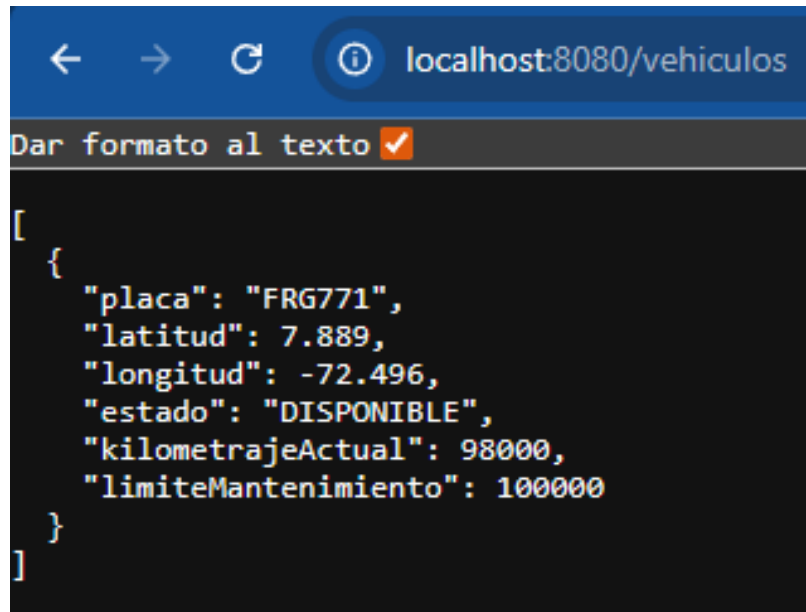
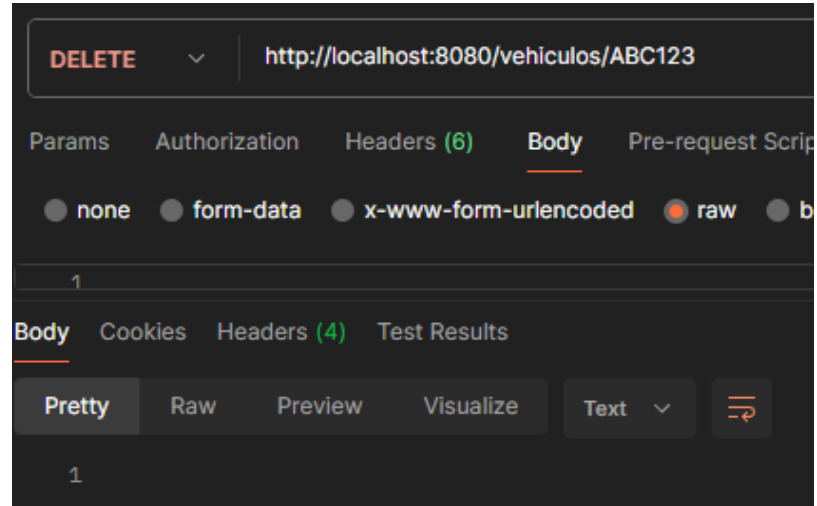
BUSCAR UN VEHICULO



A screenshot of a REST client interface. The top bar shows a 'GET' method and the URL 'http://localhost:8080/vehiculos/ABC123'. Below this, there are tabs for 'Params', 'Authorization', 'Headers (6)', 'Body', and 'Pre-request Script'. The 'Body' tab is selected, and the content is set to 'raw'. The response body is displayed in a 'Pretty' format, showing a JSON object with the same fields as the first vehicle in the previous screenshot: license plate 'ABC123', latitude 4.65, longitude -74.08, status 'DISPONIBLE', current mileage 5000.0, and maintenance limit 8500.0.

```
1 {
2   "placa": "ABC123",
3   "latitud": 4.65,
4   "longitud": -74.08,
5   "estado": "DISPONIBLE",
6   "kilometrajeActual": 5000.0,
7   "limiteMantenimiento": 8500.0
8 }
```

ELIMINAR UN VEHICULO EXISTENTE



CONCLUSIONES

El desarrollo del módulo de gestión de vehículos permitió validar la correcta aplicación de principios de diseño orientado a objetos y el uso de patrones arquitectónicos para mejorar la estructura del sistema. A lo largo de la implementación se evidenció cómo la utilización de patrones de diseño contribuye significativamente a la organización del código, la reducción del acoplamiento entre componentes y la facilidad de mantenimiento del software.

La implementación del patrón Abstract Factory dentro del módulo de vehículos permitió centralizar y abstraer el proceso de creación de los distintos tipos de vehículos del sistema. En lugar de que la lógica de negocio se encargue directamente de instanciar las clases concretas, se delega esta responsabilidad a clases especializadas encargadas de generar los objetos correspondientes. Esta estrategia mejora la modularidad del sistema y facilita la incorporación de nuevos tipos de vehículos sin necesidad de modificar la lógica existente.

Asimismo, la integración de este patrón sobre la arquitectura previamente estructurada con Factory Method demuestra cómo los patrones de diseño pueden complementarse para ofrecer soluciones más robustas y flexibles. Mientras el Factory Method define la estructura básica para la creación de objetos, el Abstract Factory permite agrupar y gestionar múltiples familias de objetos relacionados, fortaleciendo así la extensibilidad del sistema.

Las pruebas funcionales realizadas sobre las operaciones CRUD del módulo de vehículos confirmaron el correcto comportamiento de los endpoints encargados de crear, consultar, actualizar y eliminar registros dentro del sistema. Estas pruebas permitieron validar no solo el funcionamiento de las operaciones del backend, sino también la correcta interacción entre la capa de servicios, la capa de persistencia y las fábricas responsables de la creación de los objetos.

Finalmente, el uso de identificadores únicos como la placa del vehículo permitió garantizar una gestión adecuada de los registros dentro del sistema, facilitando las operaciones de búsqueda, actualización y eliminación. En conjunto, el proyecto demuestra cómo la aplicación adecuada de patrones de diseño y buenas prácticas de ingeniería de software permite construir sistemas más escalables, mantenibles y preparados para futuras extensiones funcionales.

REFERENCIAS

Freeman, E., Robson, E., Bates, B., & Sierra, K. (2004). Head First Design Patterns. O'Reilly Media. <https://www.oreilly.com/library/view/head-first-design/0596007124/>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. https://en.wikipedia.org/wiki/Design_Patterns

Oracle. (2024). Java Documentation. <https://docs.oracle.com/en/java/>

Presentación Semana 5. (2026). Implementación del patrón Abstract Factory en el módulo de vehículos. Material de clase utilizado para la explicación del diseño y funcionamiento del sistema.

Refactoring Guru. (2024). Abstract Factory Design Pattern. <https://refactoring.guru/design-patterns/abstract-factory>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003437>